



Discrete Optimization

The minimization of open stacks problem: A review of some properties and their use in pre-processing operations

Horacio Hideki Yanasse^{a,*}, Edson Luiz França Senne^b^a Instituto Nacional de Pesquisas Espaciais – INPE/LAC, Avenida dos Astronautas 1758, Jardim da Granja, 12227-010 São José dos Campos, São Paulo, Brazil^b Universidade Estadual Paulista Júlio de Mesquita Filho – UNESP, Faculdade de Engenharia de Guaratinguetá, Av. Ariberto Pereira da Cunha, 333, Portal das Colinas, 12516-410 Guaratinguetá, SP, Brazil

ARTICLE INFO

Article history:

Received 18 June 2008

Accepted 15 September 2009

Available online 19 September 2009

Keywords:

Combinatorial optimization

Minimization of open stack problem

Pre-processing

Size reduction

ABSTRACT

Pre-processing operations that reduce the size of a problem may be decisive for solving or not solving practical instances of a NP-hard problem. In this article we review some properties suggested in the literature for the minimization of open stacks problem that can be used in pre-processing operations to reduce the instances sizes. We also present a new pre-processing technique that may be very effective in reducing the size of an instance. We present computational tests with the suggested pre-processing operations applied on sets of MOSP instances of the literature and we show that the reductions obtained can be significant.

© 2009 Elsevier B.V. All rights reserved.

1. Introduction

Cutting stock problems arise in many productive settings, like in the paper, the furniture, the steel, the wooden hardboard industries. The problem consists in cutting smaller pieces (denoted items), with fixed dimensions and quantities, from larger pieces (denoted objects), with some optimization criterion, for example, maximizing profits or minimizing trim loss. A solution of a cutting stock problem is composed of a set of cutting patterns and their corresponding frequencies, that is, the number of times the patterns have to be cut in order to fulfill the demands of the desired items.

When cutting the patterns, the items cut are piled up in stacks, one stack for each item type. A stack is open the first time an item type is cut and it remains open until the last piece of the corresponding item type is cut. When the last piece is cut, the stack is closed. It is desirable to maintain a reduced number of open stacks during the cutting process because if the number of stacks increases beyond the available space, stacks must be removed in order to give space to the new stacks. Closed stacks can be removed definitively to another place or can be delivered to clients but, open stacks, if removed, must be brought back later to be completed when the corresponding item type is cut again in another pattern. This is not efficient and consumes time and resources. Therefore, there is an interest in determining a sequence to cut the patterns

that minimizes the maximum number of open stacks during the cutting process. This sequencing problem is known as the minimization of open stacks problem (MOSP).

Formally we can define MOSP as follows: given a Boolean matrix $\mathbf{C}$ in which the columns correspond to the patterns and each row corresponds to the item types, the entry c_{ij} is equal to 1 if and only if item i is contained in pattern j (the number of items i in pattern j is not relevant). Consider $\mathbf{C}_\pi$, the resulting matrix obtained by a permutation π of the columns of $\mathbf{C}$ and let $\mathbf{C}_\pi^1$ be the corresponding matrix obtained from $\mathbf{C}_\pi$ that has the consecutive 1's properties, that is, in any row of this matrix, all elements between two elements 1 is also 1. The objective is to find a permutation π of the columns, such that the maximum number of 1's in any column of matrix $\mathbf{C}_\pi^1$ is minimized.

Recently, MOSP was focused in the Constraint Modeling Challenge 2005 (see Smith and Gent, 2005) in the Fifth Workshop on Modelling and Solving Problems with Constraints. This challenge increased the interest on the MOSP by the Operations Research scientific community.

The literature on the MOSP is not extensive. Lins (1989) considered a particular case of MOSP where there are at most 2 different item types per pattern. Considering these simple patterns he built a graph associating nodes to items and arcs to patterns and obtained 1-tree graphs (i.e., a tree with an additional arc) for his practical instances. He proposed a heuristic method to solve these instances. Yuen (1991, 1995) presented six simple heuristics for MOSP. The first two heuristics use two steps, in the first, an open item is selected and in the second a pattern that contains the selected item is sequenced. In the other four heuristics the selection

* Corresponding author. Tel.: +55 12 39456551; fax: +55 12 39456375.

E-mail addresses: horacio@lac.inpe.br (H.H. Yanasse), elfsenne@fegunesp.br (E.L.F. Senne).

of the next pattern to sequence is based on a measure of “match-ing” between a pattern and the current open items. Yuen’s heuristic number 3 was considered the most efficient by the computational tests performed by the author. Yuen and Richardson (1995) presented a trivial lower bound for MOSP given by the maximum number of different item types in the patterns. They proposed the decomposition of the problem by identifying clusters of patterns (see pre-processing 1, reviewed in Section 2) and an exact method that enumerates permutations of pattern sequences. Upper bounds given by the heuristics proposed in Yuen (1991, 1995) and the new lower bound proposed are used to reduce the search space. Yanasse (1996) proposed polynomial algorithms for the 1-tree case and other graphs with very special topologies. Yanasse (1997a) presented an exact polynomial algorithm for the tree case. Yanasse (1997b) propose a non-trivial mathematical formulation for MOSP, a branch and bound algorithm and a greedy heuristic algorithm. He also presents relations of the MOSP with the Minimization of Order Spread Problem (see Garey et al., 1978; Madsen, 1988), and the minimization of the number of tool switches problem (see Tang and Denardo, 1988). Limeira (1998) and Yanasse and Limeira (1998, 2004) integrated some polynomial algorithms developed to solve special classes of instances of MOSP with the branch-and-bound algorithm proposed by Yanasse (1997b). Yanasse et al. (1998) proposed a polynomial algorithm to solve another special instance of MOSP, where the MOSP graph is a complete graph with tree subgraphs attached to its nodes. Yanasse (1997c) showed that any instance of MOSP, not only those with at most two item types per pattern as in Lins (1989), can be modeled as a graph problem. The MOSP graph is obtained as follows. Each node in the graph corresponds to an item type and there is an arc connecting two nodes if and only if there is at least one pattern containing both items types corresponding to the nodes. Therefore, a pattern with k item types is represented by a clique of size k in the MOSP graph. Observe that the MOSP graph is a union of cliques and it is completely different from the pattern graph connection suggested in Yuen and Richardson (1995). Yanasse (1997c) defined the MOSP as a graph problem as the problem of traversing the arcs of the graph such that the number of simultaneously open nodes is minimized. A node is considered closed when all the arcs incident to it are traversed, a node is considered open after the first arc incident to it is traversed. Yanasse (1997c) showed then that any MOSP instances that produce the same MOSP graph are solution-wise equivalent. In addition, since any MOSP graph can be generated by an instance of MOSP where all patterns have at most two item types, solving this particular class of MOSP instances is computationally not easier than solving any other instance of MOSP. Yanasse (1997c) also introduced two new lower bounds for MOSP given by the size of the maximal clique of the MOSP graph and the smallest degree of any node in the MOSP graph. He also showed that the maximum number of open stacks following a given sequence of patterns and its reverse sequence is the same; any MOSP having more than one pattern to sequence has multiple optimal solutions; given two instances 1 and 2 of MOSP, if 1 has a subset of the patterns of 2, then the optimal value of instance 2 is greater or equal to the optimal value of instance 1. With this last result, lower bounds for an instance of MOSP can be obtained by solving smaller MOSP instances, for instance, lower bounds can be obtained using subgraphs of the original MOSP graph. Yanasse (1997c) also showed how to obtain an optimal solution to the original problem given an optimal solution of the MOSP as a graph problem. The solution is obtained by observing, just after each arc is traversed, all the open nodes. Following the optimal sequence to traverse the arcs, the first time that all nodes (items) of a pattern of the original problem are open or closed, that pattern is sequenced. The pseudo-code of this algorithm is presented next.

Algorithm to determine an optimal solution of the original problem given an optimal solution of the MOSP graph problem

INPUT: Patterns of the original problem; Optimal sequence to traverse the arcs of the MOSP graph.

OUTPUT: Corresponding optimal sequence of the patterns.

1. S = the set of original patterns; $L(\text{arcs})$ = ordered list of arcs of the MOSP graph to traverse; $L(\text{patterns})$ = empty; OPEN = empty; CLOSED = empty.
2. While S is not empty do:
 - 2.1. Get next arc of $L(\text{arcs})$;
 - 2.2. Update OPEN with the open nodes and CLOSED with the closed nodes;
 - 2.3. Check for all patterns in S that have all their item types in set OPEN or in set CLOSED . If any, include the pattern in $L(\text{patterns})$ and delete it from S .
- Endwhile;
3. Return $L(\text{patterns})$.

The computational complexity of this algorithm is $O(m^2 n \log m)$, where m is the total number of item types and n is total number of patterns.

Faggioli and Bentivoglio (1998) presented a mathematical model for MOSP and a three phase solution method. In the first phase, a good solution is obtained with a greedy heuristic similar to some of Yuen’s (1995) heuristics. In the second phase this solution is improved using tabu search; and in the third phase, an implicit enumeration scheme of permutation of patterns is used that is just an improved procedure of Yuen and Richardson (1995). Faggioli and Bentivoglio (1998) ignore the dominated patterns in the enumeration since this does not lose the optimal solution. This is, in fact, the pre-processing operation suggested in Becceneri (1999) and in Becceneri et al. (2004) that is reviewed later in Section 2 of this manuscript, pre-processing 2.

In Yanasse et al. (1997, 1999) and Becceneri (1999), an arc contraction heuristic is presented. This arc contraction operation is also used to develop a new lower bound to the optimal value of MOSP in Yanasse et al. (1999). To the best of our knowledge, this arc contraction lower bound dominates all previous lower bounds proposed in the literature. In Becceneri (1999), the least cost node heuristic for MOSP was proposed and it was later modified in Becceneri et al. (2004). In this heuristic, the next arcs of the MOSP graph to be traversed are chosen by closing the node with the least number of arcs incident to it. To the best of our knowledge, the performance of this heuristic, in terms of the quality of the solution, is the best or among the best of the literature. Ashikaga (2001) presented two new heuristics for solving the MOSP. In the first he searches for Hamiltonian circuits and for that, he uses a variation of the extension–rotation algorithm of Pósa (1976). This algorithm starts with a path with a single node and tries to extend it as far as possible. Ashikaga’s (2001) variation consisted in choosing the set of nodes of the maximal clique of the MOSP graph, instead of a single node and to restrict the extensions. The maximal clique is determined by a greedy search for the minimum grade vertex of the complementary MOSP graph. The second heuristic proposed by Ashikaga (2001), uses a recursive contraction of maximal cliques until a single hyper-node is obtained. By the computational tests performed, Ashikaga’s first heuristic was better than Yuen’s heuristic 3, in terms of quality of solution obtained and smaller execution times. Meta-heuristics were also suggested for solving the MOSP. Linhares et al. (1999) developed a simulated annealing heuristic, Fink and Voss (1999) used simulated annealing and tabu search, Oliveira and Lorena (2002a,b) used a constructive genetic algorithm. It is worth observing that Linhares (2001) and Oliveira and Lorena (2002a) reported better solutions for some data set of instances compared to the solutions presented by Faggioli and

Bentivoglio's (1998) exact methods. This indicates that the implementation of the algorithm reported in Faggioli and Bentivoglio's (1998) was faulty. In Lamosa (2003) the performance of a parallel implementation of Yanasse's (1997) exact algorithm was reported. In Becceneri et al. (2004) an improved implementation of Yanasse's (1997) algorithm was reported with the use of upper bounds given by an arc contraction heuristic. In Yanasse et al. (2002a, 2007) exact and heuristic methods are proposed by using properties of the solution of MOSP to establish partial orders in which the nodes in the graph should be closed. Equivalent nodes (reviewed in pre-processing operations 5 and 6 in Section 2) and adjacent nodes with degree 2 (reviewed in pre-processing 3 in Section 2) are pointed out. Banda and Stuckey (2007) present a dynamic-programming solution to MOSP, simplifying the search using properties presented in Becceneri et al. (2004) and Yuen and Richardson (1995).

There are many similar but not equivalent optimization problems to MOSP (see Linhares and Yanasse, 2002), like the Minimization of Order Spread Problem (see Foerster and Wäscher, 1998; Fink and Voss, 1999), the Matrix Bandwidth Minimization (see Chinn et al., 1982; Lim et al., 2006), or the Minimization of the Discontinuities Problem (see Dyson and Gregory, 1974; Madsen, 1988). Also, there are a set of equivalent problems to MOSP that arise in completely different contexts, like the Gate Matrix Layout Problem, the One dimensional logic, and PLA folding in VLSI design, Interval Thickness, Node Search Game, Edge Search Game, Narrowness, Split Bandwidth, Graph path-width, Edge Separation, and Vertex Separation in graph theory (see Linhares and Yanasse, 2002; Möhring, 1990).

From the report of the Constraint Modeling Challenge 2005, and more recent papers dealing with MOSP (see, for instance, Banda and Stuckey (2007)) it seems that the existing results of the literature about the MOSP are not being explored on all their potential. Therefore, in this work we review some pre-processing operations proposed in the literature for the MOSP and discuss how some properties derived for MOSP can be used on additional pre-processing operations.

2. Pre-processing operations

2.1. Pre-processing 1: Problem decomposition

MOSP is decomposable when there are independent clusters of patterns. Independent clusters of patterns can be found using graphs (see Yuen and Richardson, 1995). Consider, for example, the instance of MOSP given in Table 1.

Build a pattern connection graph of this instance. A pattern connection graph is obtained as follows: each pattern corresponds to a vertex, and there exists an arc linking two vertices in this graph if and only if the patterns corresponding to these two vertices share at least one item type in common. In the pattern connection graph, if two nodes are not adjacent, then their corresponding patterns are not connected, hence, the stacks they form have no relation to each other.

The pattern connection graph of instance 1 is presented in Fig. 1. We observe that patterns P_1 , P_2 and P_3 have item types in

Table 1
Instance 1 with 5 patterns and 8 item types.

Patterns	Types of items
P_1	1 7
P_2	3 6 7
P_3	7 8
P_4	5
P_5	2 4

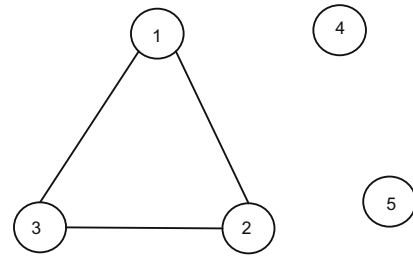


Fig. 1. Pattern connection graph of instance 1.

common, while the items in patterns P_4 and P_5 just appear in these patterns. So, we identify 3 clusters of patterns (items) that can be treated independently for the MOSP.

In the following, we will assume, as before, that m is the number of item types and n is the number of patterns in a MOSP instance. The verification of the connectivity of the graph can be performed, in the worst case, with $O(n^2)$ operations.

We assume that we have already performed pre-processing 1 when considering the other pre-processing operations.

2.2. Pre-processing 2: Reducing the number of patterns due to dominance

A dominated pattern is one whose item types are all contained in another pattern of the problem. Dominated patterns can be withdrawn from the problem, since they can be sequenced afterwards, immediately after its dominant pattern is sequenced, without increasing the number of open stacks.

The following pre-processing operations are derived from properties observed by analyzing the MOSP graph (see Yanasse, 1997c). Dominance of patterns was used to reduce the search in the solution space, for instance, in Yuen and Richardson (1995), Becceneri et al. (2004), Banda and Stuckey (2007).

To identify dominances between all the patterns, it is necessary to compare all pair of patterns requiring an effort of at most $O(n^2m)$.

2.3. Pre-processing 3: Reducing the number of items due to the simplicity of the patterns that contain them

If there exists two adjacent nodes i and j in the MOSP graph, both having degree 2 then, to any feasible order in which the nodes are closed, it is possible to construct an alternative corresponding sequence, whose number of open nodes is less than or equal to the one obtained with the original sequence and where node j is closed immediately before node i is opened or node i is closed immediately before node j is opened (see Yanasse et al., 2002b).

For the sake of finding a solution to MOSP, it is possible to collapse the arc (i, j) of the MOSP graph, obtaining a reduced graph with one node less, if the resulting node maintains the degree 2. This way we can downsize the problem in terms of total number of item types.

To identify pair of adjacent nodes having both degree 2, in the worst case, we have to verify all pair of vertices, implying a complexity of at most $O(m^2)$.

2.4. Pre-processing 4: Reducing the number of patterns by determining an equivalent instance

Consider the MOSP instances 2 and 3 given in Tables 2 and 3, respectively.

The MOSP graph corresponding to instances 2 and 3 is presented in Fig. 2. We observe that different MOSP instances generate

Table 2
Instance 2 with 8 patterns and 6 item types.

Patterns	Types of items
P_1	1 2 4
P_2	2 4 5
P_3	3 4
P_4	1 3 5
P_5	1 4 5
P_6	5 6
P_7	4 6
P_8	2 6

Table 3
Instance 3 with 6 patterns and 6 item types.

Patterns	Types of items
P_1	1 2 4
P_2	2 4 5 6
P_3	3 4
P_4	1 3 5
P_5	1 4 5
P_6	5 6

the same MOSP graph. The optimal solution to the MOSP graph is the same for both problems. Therefore, these instances are computationally equivalent in terms of their solution. It is worth remembering that from the optimal solution of the MOSP graph we can determine, in polynomial time, a corresponding optimal solution to the original instances 2 and 3 (see Yanasse, 1997c) using the algorithm described in the previous section.

This equivalence of solutions led us to find alternative smaller instances of MOSP that produces the same MOSP graph of a given instance. Since MOSP is NP-hard, the smaller the alternative instance, the better, since the computational time to solve the problem increases with its size.

In pre-processing 4 we propose an algorithm, denoted by REDUCE_PATTERNS, that generates an alternative potentially smaller sized MOSP instance that is equivalent to the original instance in terms of the MOSP graph.

Algorithm REDUCE_PATTERNS

Given: Instance of MOSP

Initialization: Generate the MOSP graph.

Classify all arcs as type 1.

Main Step: While there are arcs type 1 in the graph do

Select among the remaining original patterns the one corresponding to a clique with the largest number of arcs type 1; break ties at random;

Determine a maximal clique containing this pattern. Store the clique (pattern);

Classify all arcs of this clique as type 2.

Endwhile.

Illustration of Algorithm REDUCE_PATTERNS

Let us apply algorithm REDUCE_PATTERNS to instance 2 given in Table 2. The corresponding MOSP graph is given in Fig. 2.

In the first pass of the while loop of REDUCE_PATTERNS, we have that all arcs are type 1. There are many patterns tied: P_1 , P_2 , P_4 and P_5 . Let us select, for instance, pattern P_2 . The complete subgraph composed of nodes {2,4,5,6} is a maximal clique containing pattern P_2 . Let Q_1 be the pattern containing the item types [2 4 5 6]. The resulting graph, after classifying the arcs of this clique as type 2 arcs, is illustrated in Fig. 3. Note that node 6 is a “covered node” because only arcs type 2 are incident to node 6. Covered nodes do not need to be considered furthermore in the subsequent stages of the algorithm.

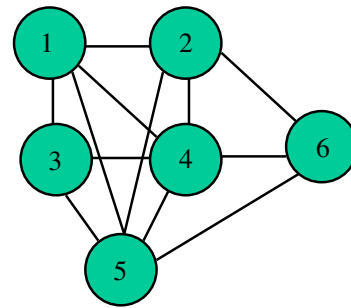


Fig. 2. MOSP graph of instances 2 and 3.

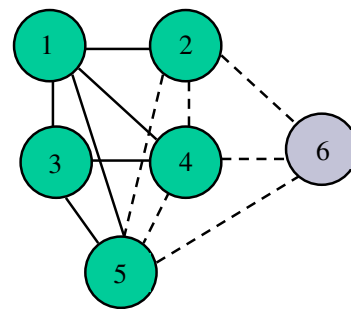


Fig. 3. Graph after first iteration of the while loop in REDUCE_PATTERNS.

Table 4
Remaining original patterns with arcs type 1 left.

Patterns	Types of items
P_1	1 2 4
P_3	3 4
P_4	1 3 5
P_5	1 4 5

In Table 4 we present the remaining original patterns with arcs type 1 left.

The next pattern to select is pattern P_5 that has the largest number of arcs type 1 in its corresponding clique in the MOSP graph. The complete subgraph composed of nodes {1,3,4,5} is a maximal clique containing pattern P_5 . Let Q_2 be the pattern containing the item types [1 3 4 5]. The resulting graph, after classifying the arcs of this clique as type 2 arcs, is illustrated in Fig. 4.

In Table 5 we present the remaining original patterns with arcs type 1 left.

The single pattern left is P_1 and a maximal clique containing this pattern is the complete subgraph composed of nodes {1,2,4,5}. Let

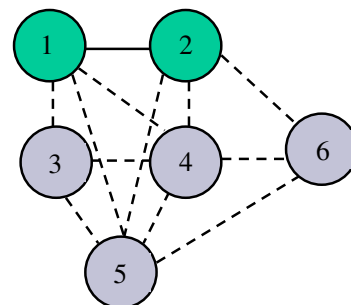


Fig. 4. Graph after second iteration of the while loop in REDUCE_PATTERNS.

Table 5

Remaining original patterns with arcs type 1 left.

Patterns	Types of items
P_1	1 2 4

Table 6

Equivalent MOSP problem to instances 2 and 3 with 3 patterns and 6 item types.

Patterns	Types of items
Q_1	2 4 5 6
Q_2	1 3 4 5
Q_3	1 2 4 5

Q_3 be the pattern containing the item types [1 2 4 5]. In the resulting graph, after classifying the arcs of this clique as type 2 arcs, there are no arcs of type 1 left and we are finished. The equivalent MOSP problem obtained is given in Table 6.

So, we were able to reduce an instance of MOSP with 8 patterns to an equivalent one with only 3 patterns.

Algorithm REDUCE_PATTERNS tries to determine an equivalent MOSP instance with a smaller number of patterns. The new patterns are generated based on the original patterns, therefore, it is easy to prove that the number of resulting patterns of the algorithm is at most the number of original patterns. So, in the worst case, we are left with the original patterns.

The actual problem we are interested in solving is to determine an equivalent problem with the smallest number of patterns, or equivalently, the smallest number of cliques that covers all arcs of the graph. This seems to be itself a difficult problem to solve. Algorithm REDUCE_PATTERNS is a simple heuristic proposed to solve this problem that does not require too much computational effort.

The computational complexity of algorithm REDUCE_PATTERNS is $O(m^3n)$. In the Main Step of algorithm REDUCE_PATTERNS, for each pattern (or corresponding clique) we have to verify how many arcs type 1 it contains. This can be done in at most $O(m^2)$ operations. Since this must be done for all patterns, in the worst case, we have at most $O(m^2n)$ operations. To determine a maximal clique containing a pattern, we propose the following heuristic that needs at most $O(m^3)$ operations:

Algorithm MaxClique containing a pattern

INPUT: MOSP graph and a pattern (that is, the set of nodes of the pattern).

OUTPUT: Maximal clique that contains the pattern (it may not be optimal).

A = set of nodes of the pattern;

B = empty set;

$k = 0$;

Let G^k be the current graph.

While the degrees of all nodes in set A of graph G^k are not equal do

 Select any node in A, say node a, with the smallest degree in G^k . Delete from G^k all nodes that are not adjacent to a, obtaining graph G^{k+1} ;

$k \leftarrow k+1$;

Endwhile

Insert in B the set of nodes in G^k that are not in A.

If B is empty then return set A and STOP.

While the degrees of all nodes in graph G^k are not equal to $|G^k| - 1$ do

 Delete from G^k a node in B with the smallest degree, obtaining graph G^{k+1} ;

$k \leftarrow k + 1$;

Endwhile

Update A with all nodes of G^k .

Return set A and STOP.

In the worst case this must be done for all patterns, hence, we have at most $O(m^3n)$ operations.

2.5. Pre-processing 5: Reducing the number of items due to “pseudo equivalence”

Let A_i be the set of nodes in the MOSP graph that are adjacent to node i . Let i and j be nodes of a MOSP graph such that $\{A_i \cup \{i\}\} = \{A_j \cup \{j\}\}$. Then there exists always an optimal solution to MOSP where i and j are closed one immediately after the other. Using this result, we can reduce the number of item types, by considering nodes i and j as “twin” nodes, that is, they are two but they are always together as if they are a single node (they are open together, they are closed together).

Consider the MOSP graph instance given in Fig. 2. In this MOSP graph, $\{A_4 \cup \{4\}\}$ and $\{A_5 \cup \{5\}\}$ are equal. Therefore, items 4 and 5 can be consolidated into a single “twin node” (4 & 5) (see Fig. 5).

To identify twin nodes, in the worst case, we have to verify all pair of vertices, implying a complexity of at most $O(m^3)$.

Without loss of generality, let us consider now that the MOSP instance has been reduced using algorithm REDUCE_PATTERNS (pre-processing 4).

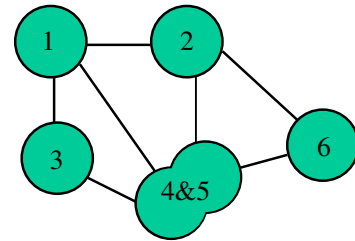


Fig. 5. Reduced instance of MOSP.

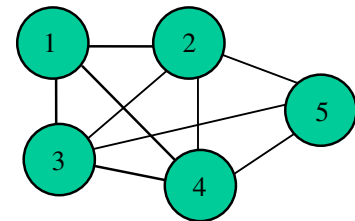


Fig. 6. Instance 4 of MOSP.

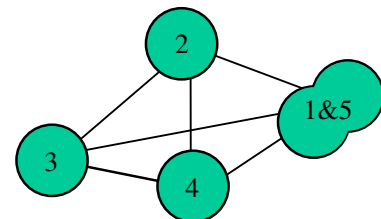


Fig. 7. Reduced instance of MOSP.

$Q^5 = [2\ 3\ 4\ 5]$ that are part of the patterns obtained with algorithm REDUCE_PATTERNS, can be consolidated into a single “twin-pattern” $Q^{1\&5} = [1\ 2\ 3\ 4] \& [2\ 3\ 4\ 5]$ (see Fig. 7).

Similarly to pre-processing 5, to identify pseudo equivalent nodes, in the worst case, we have to verify all pair of vertices, implying a complexity of at most $O(m^3)$.

2.7. Pre-processing 7: Polynomial solvable instances of MOSP

There are many instances of MOSP that can be solved exactly by polynomial algorithms.

Some easy cases can be easily identified using the MOSP graph. MOSP graphs that are trees, one-trees, complete graphs, are easy to solve and to identify (see Yanasse, 1996, 1997b). Many other poly-

nomially solvable cases can be derived using special structured MOSP graphs. However, verifying whether an instance falls in one of these cases may require a computational effort that may be not worthy; it may be better to solve the original problem itself by some general method.

The computational complexity to identify trees, one-trees or complete graphs is bounded by $O(m)$, $O(m)$ and $O(m^2)$, respectively.

3. Computational experiments

The proposed pre-processing operations were applied to a set of MOSP problems proposed and used in the literature. These sets were proposed in Smith and Gent (2005). In Table 7 we present

Table 9

Average execution times (in seconds) to perform the pre-processing operations.

Author	File	Size	# Inst	PP1	PP2	PP3	PP4	PP5	PP6	PP7
Smith	10_10_35.txt	(10,10)	5	0.01	0.00	0.00	0.00	0.00	0.00	0.00
	10_20_35.txt	(10,20)	5	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	15_30_35.txt	(15,30)	5	0.00	0.00	0.00	0.01	0.00	0.01	0.00
Harvey	wbo_10_10.txt	(10,10)	40	0.02	0.00	0.00	0.00	0.00	0.00	0.00
	wbo_10_20.txt	(10,20)	40	0.00	0.00	0.00	0.03	0.02	0.02	0.00
	wbo_10_30.txt	(10,30)	40	0.01	0.02	0.02	0.05	0.06	0.05	0.00
	wbo_15_15.txt	(15,15)	60	0.00	0.00	0.00	0.02	0.01	0.00	0.00
	wbo_15_30.txt	(15,30)	60	0.01	0.02	0.01	0.06	0.05	0.06	0.01
	wbo_20_10.txt	(20,10)	70	0.00	0.02	0.02	0.02	0.02	0.00	0.00
	wbo_20_20.txt	(20,20)	90	0.01	0.00	0.03	0.02	0.02	0.01	0.00
	wbo_30_10.txt	(30,10)	100	0.02	0.03	0.00	0.00	0.01	0.00	0.00
	wbo_30_15.txt	(30,15)	120	0.03	0.03	0.02	0.05	0.00	0.00	0.00
	wbo_30_30.txt	(30,30)	140	0.06	0.05	0.03	0.16	0.12	0.06	0.03
	wbop_10_10.txt	(10,10)	40	0.00	0.00	0.00	0.00	0.00	0.02	0.00
	wbop_10_20.txt	(10,20)	40	0.01	0.00	0.01	0.02	0.00	0.00	0.00
	wbop_10_30.txt	(10,30)	40	0.00	0.02	0.00	0.06	0.01	0.03	0.00
	wbop_15_15.txt	(15,15)	60	0.02	0.00	0.02	0.00	0.02	0.02	0.00
	wbop_15_30.txt	(15,30)	60	0.00	0.01	0.00	0.06	0.01	0.03	0.00
	wbop_20_10.txt	(20,10)	40	0.01	0.00	0.01	0.00	0.00	0.00	0.00
	wbop_20_20.txt	(20,20)	90	0.02	0.01	0.00	0.03	0.03	0.02	0.00
	wbop_30_10.txt	(30,10)	40	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	wbop_30_15.txt	(30,15)	60	0.03	0.02	0.00	0.00	0.02	0.01	0.00
	wbop_30_30.txt	(30,30)	140	0.06	0.06	0.02	0.17	0.16	0.09	0.00
	wbp_10_10.txt	(10,10)	40	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	wbp_10_20.txt	(10,20)	70	0.00	0.00	0.00	0.03	0.00	0.02	0.00
	wbp_10_30.txt	(10,30)	100	0.02	0.02	0.00	0.05	0.03	0.05	0.00
	wbp_15_15.txt	(15,15)	60	0.02	0.00	0.00	0.02	0.00	0.01	0.00
	wbp_15_30.txt	(15,30)	120	0.01	0.03	0.00	0.09	0.05	0.06	0.00
	wbp_20_10.txt	(20,10)	40	0.00	0.00	0.00	0.01	0.00	0.00	0.00
	wbp_20_20.txt	(20,20)	90	0.03	0.02	0.01	0.01	0.03	0.00	0.02
	wbp_30_10.txt	(30,10)	40	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	wbp_30_15.txt	(30,15)	60	0.01	0.00	0.00	0.02	0.01	0.02	0.00
	wbp_30_30.txt	(30,30)	140	0.06	0.05	0.06	0.16	0.11	0.05	0.00
Miller Shaw	Miller19.txt	(20,40)	1	0.00	0.00	0.00	0.01	0.00	0.00	0.00
	ShawInstances.txt	(20,20)	25	0.00	0.00	0.02	0.00	0.00	0.00	0.00
Simonis	problem_10_10.txt	(10,10)	550	0.03	0.00	0.00	0.00	0.03	0.02	0.00
	problem_10_20.txt	(10,20)	550	0.06	0.02	0.02	0.11	0.11	0.03	0.00
	problem_15_15.txt	(15,15)	550	0.03	0.02	0.03	0.11	0.03	0.02	0.00
	problem_15_30.txt	(15,30)	220	0.05	0.06	0.06	0.11	0.19	0.06	0.00
	problem_20_10.txt	(20,10)	550	0.09	0.03	0.01	0.02	0.00	0.02	0.00
	problem_20_20.txt	(20,20)	220	0.03	0.03	0.00	0.08	0.06	0.02	0.00
	problem_30_10.txt	(30,10)	550	0.09	0.08	0.03	0.03	0.03	0.01	0.00
	problem_30_15.txt	(30,15)	220	0.05	0.03	0.00	0.02	0.03	0.00	0.00
	problem_30_30.txt	(30,30)	110	0.05	0.02	0.06	0.03	0.14	0.01	0.00
	problem_40_20.txt	(40,20)	110	0.06	0.03	0.01	0.05	0.03	0.02	0.00
Wilson	gp50by50.txt	(50,50)	4	0.00	0.00	0.01	0.00	0.05	0.00	0.00
	gp100by100.txt	(100,100)	4	0.06	0.02	0.03	0.02	0.56	0.00	0.00
	nwrsmaller4a.txt	(10,20)	2	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	nwrsmaller4b.txt	(15,25)	2	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	nwrslarger4a.txt	(20,30)	2	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	nwrslarger4b.txt	(25,60)	2	0.00	0.00	0.00	0.02	0.00	0.00	0.00
	sp1.txt	(25,25)	1	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	sp2.txt	(50,50)	1	0.00	0.00	0.00	0.00	0.00	0.02	0.00
	sp3.txt	(75,75)	1	0.00	0.00	0.00	0.01	0.00	0.00	0.00
	sp4.txt	(100,100)	1	0.00	0.02	0.00	0.03	0.00	0.01	0.00

the reduction obtained with each pre-processing operation suggested. In this table, columns PP1 to PP7 indicate the pre-processing operations 1–7. The size of an instance is given in terms of the pair (number of patterns, number of item types). After each pre-processing operation this pair corresponds to (average number of patterns, average number of item types). By # Inst we mean the number of instances in the file. The notation T , $1 - T$ and CG in column PP7 means tree, one-tree and complete graph, respectively. The numbers below columns T , $1 - T$ and CG in column PP7 counts the total number of instances observed in which the corresponding MOSP graph was a tree, one-tree, and a complete graph, respectively.

From the computational tests we can observe an overall reduction in size of many sets of instances. Some of the reductions are impressive. For example, all the instances in sets gp50by50 and gp100by100 were reduced to instances with a single pattern.

For these set of instances, pre-processing operation 4 was the one that reduced most the size of the problems.

The pre-processing operations of Section 2 do not require a high computational effort. Table 8 provides an overall summary of the time complexity for the pre-processing operations, in which m is the number of item types and n is the number of patterns.

In Table 9 we present the average times in seconds to perform the pre-processing operations to sets of problems of the literature. As can be seen, the longest time is less than 0.6 seconds.

4. Concluding remarks

In this work we presented some pre-processing operations that can be performed to reduce the size of MOSP instances. MOSP is NP-hard, and in hard combinatorial problems where the execution time of an algorithm to get a solution increases very quickly with the size of the problem, it is always of great interest performing problem data pre-processing to simplify or reduce the size of the problem.

Although some of the results are not recent, it seems that they are not known or they have been ignored by many researchers working on solution procedures for the MOSP. Pre-processing 1 and 2 were previously proposed in the literature, pre-processing operations 3, 5, 6 and 7 are derived from earlier properties of the MOSP problem defined as a graph problem, and algorithm REDUCE_PATTERNS proposed in pre-processing 4 is a new contribution being presented in this article. Algorithm REDUCE_PATTERNS presented in this article has an advantage over a previous algorithm proposed in Yanasse (2008) since it has the guarantee of always producing instances with at most the same number of patterns of the original instances.

We believe that all pre-processing operations and, in particular, pre-processing 4, are a relevant contribution for those interested in solving MOSP because the size reduction accomplished with these operations are significant as can be observed from the extensive computational test results obtained.

We observe that reduction in size does not necessary implies an increase in resolution simplicity. It depends on the approach used to solve the problem. As it can be seen from the pre-processing strategies, there are characteristics of the instances that may strongly impact on their resolution by optimization methods. Interested readers and researchers working with MOSP should be aware of these characteristics and, apparently, from the published literature, many of them are not. One can observe that the great majority of the authors solve MOSP without considering all the particular characteristics of the instances, at least the ones pointed out in this manuscript. Many test instances of the Constraint Modelling Challenge 2005, for instance, can be significantly reduced in size and, consequently, the execution times of the algorithms pro-

posed by the participants of this competition, would be significantly reduced. All the heuristic methods of the literature that search for permutations of patterns can also have an improved performance with a smaller number of patterns. Therefore, this manuscript is useful to inform this aspect. For those that already work directly (or indirectly) with the MOSP graph there is no benefit in reducing the number of patterns in an instance since the MOSP graph is the same independent of the number of patterns.

Acknowledgements

This work is partially financed by CNPq and FAPESP. We would like to thank the suggestions of the anonymous referees that improved the presentation of this article.

References

- Ashikaga, F.M., 2001. Um método frugal para o problema de minimização de pilhas abertas, Dissertação, Mestrado em Ciência no Curso de Engenharia Eletrônica e Computação na Área de Informática, ITA/CTA, São José dos Campos.
- Banda, M.G., Stuckey, P.J., 2007. Dynamic programming to minimize the maximum number of open stacks. *INFORMS Journal on Computing* 19 (4), 607–617.
- Becceneri, J.C., 1999. O problema de seqüenciamento de padrões para a minimização do número máximo de pilhas abertas em ambientes de cortes industriais, Tese, Doutorado em Ciências no Curso de Engenharia Eletrônica e Computação na Área de Informática, ITA/CTA, São José dos Campos.
- Becceneri, J.C., Yanasse, H.H., Soma, N.Y., 2004. A method for solving the minimisation of the maximum number of open stacks problem within a cutting process. *Computers and Operations Research* 31 (14), 2315–2332.
- Chinn, P., Chvatalova, J., Dewdney, A.K., Gibbs, N.E., 1982. The bandwidth problem for graphs and matrices – a survey. *Journal of Graph Theory* 6, 223–254.
- Dyson, R.G., Gregory, A.S., 1974. The cutting stock problem in the flat glass industry. *Operational Research Quarterly* 25 (1), 41–53.
- Faggioli, E., Bentivoglio, C.A., 1998. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research* 110 (3), 564–575.
- Fink, A., Voss, S., 1999. Applications of modern heuristic search methods to pattern sequencing problems. *Computers and Operations Research* 26, 17–34.
- Foerster, H., Wäscher, G., 1998. Simulated annealing for order spread minimization in sequencing cutting patterns. *European Journal of Operational Research* 110, 272–281.
- Garey, M.R., Graham, R.L., Johnson, D.S., Knuth, D.E., 1978. Complexity results for bandwidth minimization. *SIAM Journal of Applied Mathematics* 34 (3), 477–495.
- Lamosa, D.M., 2003. Implementação paralela de um algoritmo exato para a resolução de um problema de seqüenciamento de padrões de corte, Dissertação, Mestrado em Computação Aplicada, INPE, São José dos Campos.
- Limeira, M.S., 1998. Desenvolvimento de um algoritmo exato para a solução de um problema de seqüenciamento de padrões de corte, Dissertação, Mestrado em Computação Aplicada, Instituto Nacional de Pesquisas Espaciais, São José dos Campos.
- Lim, A., Rodrigues, B., Xiao, F., 2006. Heuristics for matrix bandwidth reduction. *European Journal of Operational Research* 174, 69–91.
- Linhares, A., 2001. Industrial pattern sequencing problems: Some complexity results and new local search models, Tese, Doutorado em Computação Aplicada, Instituto Nacional de Pesquisas Espaciais, São José dos Campos.
- Linhares, A., Yanasse, H.H., 2002. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers and Operations Research* 29 (12), 1759–1772.
- Linhares, A., Yanasse, H.H., Torreão, J.R.A., 1999. Linear gate assignment: A fast statistical mechanics approach. *IEEE Transaction on Computer Aided Design of Integrated Circuits and Systems* 18 (12), 1750–1758.
- Lins, S., 1989. Traversing trees and scheduling tasks for duplex corrugator machines. *Pesquisa Operacional* 9 (1), 40–54.
- Madsen, O.B.G., 1988. An application of travelling-salesman routines to solve pattern-allocation problems in the glass industry. *Journal of the Operational Research Society* 39 (3), 249–256.
- Möhring, R.H., 1990. Graph problems related to gate matrix layout and PLA folding. *Computing* 7, 17–51.
- Oliveira, A.C.M., Lorena, L.A.N., 2002a. A constructive genetic algorithm for gate matrix layout problems. *IEEE Transactions On Computer-Aided Design of Integrated Circuits and Systems* 21 (8), 969–974.
- Oliveira, A.C.M., Lorena, L.A.N., 2002. 2-Opt population training for minimization of open stack problem. In: Bittencourt, G., Ramalho, G.L. (Eds.), *Advances in Artificial Intelligence*, Springer Lecture Notes in Artificial Intelligence Series, vol. 2507, Springer, Berlin, pp. 313–323.
- Pósa, L., 1976. Hamiltonian circuits in random graphs. *Discrete Mathematics* 14, 359–364.

- Smith, B.M., Gent, I.P., 2005. Constraint modelling challenge. In: *The Fifth Workshop on Modelling and Solving Problems with Constraints, IJCAI 05, Edinburgh*. <<http://www.dcs.st-and.ac.uk/~ipg/challenge/index.html>>, <<http://www.dcs.st-and.ac.uk/~ipg/challenge/ModelChallenge05.pdf>>.
- Tang, C.S., Denardo, E.V., 1988. Models arising from a flexible manufacturing machine, Part I: Minimization of the number of tool switches. *Operations Research* 36 (5), 767–777.
- Yanasse, H.H., 1996. Minimization of open orders-polynomial algorithms for some special cases. *Pesquisa Operacional* 16 (1), 1–26.
- Yanasse, H.H., 1997a. An exact algorithm for the tree case of the minimization of open orders problem. *XXIX Simpósio Brasileiro de Pesquisa Operacional (SBPO)*, Salvador, BA.
- Yanasse, H.H., 1997b. On a pattern sequencing problem to minimize the maximum number of open stacks. *European Journal of Operational Research* 100, 454–463.
- Yanasse, H.H., 1997c. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional* 17 (1), 57–70.
- Yanasse, H.H., 2008. Pre-processing operations for the minimization of open stacks problem. in: *Proceedings (in CD) of the XL SBPO, XL SBPO-Simpósio Brasileiro de Pesquisa Operacional, João Pessoa, arq0249, SE, Brazil, September 2–5, pp. 1780–1789. ISSN:1518-1731*.
- Yanasse, H.H., Limeira, M.S., 1998. Um algoritmo branch-and-bound para resolver um problema de sequenciamento de padrões. In: *XXX SBPO-Simpósio Brasileiro de Pesquisa Operacional, Curitiba, Paraná*.
- Yanasse, H.H., Limeira, M.S., 2004. Refinements on an enumeration scheme for solving a pattern sequencing problem. *International Transactions in Operations Research* 11, 277–292.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., (1998). A collapsing arc heuristic for solving a patterns sequencing problem. In: *Proceedings (in CD) 17º Encontro Nacional de Engenharia de Produção/Third International Congress of Industrial Engineering, Gramado, RS, October 6–9*.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 1998. Um algoritmo exato para um caso especial do problema de sequenciamento de padrões. *XXX SBPO-Simpósio Brasileiro de Pesquisa Operacional, Curitiba, PR*.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 1999. Bounds for a problem of sequencing patterns. *Pesquisa Operacional* 19 (2), 249–277.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 2002a. A heuristic to solve a pattern sequencing problem based on partial orderings. In: *XXII Encontro Nacional de Engenharia de Produção/ VIII International Conference on Industrial Engineering and Operations Management, Curitiba, PR*.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 2002b. Ordenamento parcial para reduzir o espaço de busca de uma solução ótima para um problema de sequenciamento de padrões. In: *XXXIV Simpósio Brasileiro de Pesquisa Operacional, Rio de Janeiro, RJ*.
- Yanasse, H.H., Becceneri, J.C., Soma, N.Y., 2007. Um algoritmo exato com ordenamento parcial para solução de um problema de programação da produção: Experimentos computacionais. *Gestão & Produção* 14 (2), 353–361.
- Yuen, B.J., 1991. Heuristics for sequencing cutting patterns. *European Journal of Operational Research* 55, 183–190.
- Yuen, B.J., 1995. Improved heuristics for sequencing cutting patterns. *European Journal of Operational Research* 87, 57–64.
- Yuen, B.J., Richardson, K.V., 1995. Establishing the optimality of sequencing heuristics for cutting stock problems. *European Journal of Operational Research* 84, 590–598.